CECS 327 Distributed Lodging — Milestone 2: Communication Mechanisms Report

Course: CECS 327, Section 02, CSULB **Repository:** https://github.com/haileyc1201/CECS327-Distributed-Lodging **Branch:** `main` (Python simulation)

1. Overview

The Python simulation on `main` uses the message envelope and wire formats described below.

- Each node is its own OS process: Property RPC, Reservation RPC, a pub-sub Broker, a Group Coordinator, 3 subscriber services and 3 property replicas. They talk over **raw TCP sockets** with length-prefixed frames carrying **JSON-RPC 2.0** and event messages. It builds on Hailey's Milestone 1 Property/Reservation socket services and the same `data/properties.json`. Her original services and client still run unchanged.

2. Communication model

Aspect	Python simulation (`main`)
Transport	TCP sockets (`socket` module), 4-byte length + 1-byte format header per frame
Processes / nodes	10 OS processes: property-rpc :6001, reservation-rpc :6002, broker :6010, group-coordinator :6020, 3 subscribers, 3 replicas :6101-6103
Synchronous style	Blocking request-reply on a persistent socket
Asynchronous style	Broker pushes `event` frames over long-lived subscriber sockets
Shared state	In-memory per node; property data from `data/properties.json` (not modified by demo)

Booking flow: **client → Reservation (RPC) → Property `availability.check` (request-reply) → mark unavailable → publish `booking.created` (pub-sub) → multicast `availability_update` to group `property-replicas` (group communication) → reply to client** listing which subscribers and replicas were reached.

3. Message formats

Every message is an **envelope**:

```
{
  "id": "fc94cb9cacac",
  "type": "rpc.request",
```

```
  "sender": "client",
  "receiver": null,
  "timestamp": "2026-10-12T23:16:20.185Z",
  "correlation_id": null,
  "payload": {"jsonrpc": "2.0", "method": "reservation.book",
              "params": {"property_id": 101, "guest_name": "Demo Guest", "nights":
2}, "id": 3}
}
```

The reply carries `correlation_id` = the request's `id`, which lets clients match replies to requests:

```
{"id":"a13de2f6078d","type":"rpc.reply","sender":"reservation-
rpc","receiver":"client",
 "timestamp":"2026-10-12T23:16:20.195Z","correlation_id":"fc94cb9cacac",
 "payload":{"jsonrpc":"2.0","id":3,"result":{"status":"accepted",
   "reservation":{"reservation_id":"R-
27d48c43","property_id":101,"amount":300,"status":"confirmed"},
   "notified_subscribers":["notification-service","analytics-service","audit-
service"],
   "replicas_updated":["replica-1","replica-2","replica-3"]}}}
```

Pub-sub event and group multicast payloads:

```
{"type":"event","sender":"broker","payload":{"topic":"booking.created","seq":1,
  "publisher":"reservation-rpc","event":{"reservation_id":"R-
27d48c43","property_id":101,"amount":300}}}

{"type":"group.deliver","sender":"group-coordinator","receiver":"replica-
2","payload":{
  "group":"property-replicas","seq":1,"origin":"reservation-rpc",
  "message":{"op":"availability_update","property_id":101,"available":false}}}
```

Message types: `rpc.request` / `rpc.reply`, `subscribe` / `subscribe.ack`, `publish` / `publish.ack`, `event` / `event.ack`, `group.join` / `group.leave` / `group.members` / `group.multicast` (+ `.ack` / `.reply`), `group.deliver`, `group.view`, `group.ack`, `group.heartbeat`.

Second format: typed XML. The same envelope can be sent as XML. Each element carries a type attribute (`t="int|float|bool|str|null|dict|list"`), so the round-trip is lossless:

```
<message><envelope t="dict"><entry key="id"><value t="str">9aee0f075a81</value>
</entry>
<entry key="type"><value t="str">rpc.request</value></entry> ...
<entry key="payload"><value t="dict"><entry key="params"><value t="dict">
<entry key="property_id"><value t="int">103</value></entry></value></entry></value>
</entry></envelope></message>
```

4. Marshalling and unmarshalling

Step	Python (`src/communication/codec.py`, `transport.py`)
Marshal	`marshal(envelope, "json"/"xml")` → UTF-8 bytes (`json.dumps` / ElementTree)
Framing	`[len:4][fmt:'J'/'X'][body]`. Removes Milestone 1's 1024-byte `recv` limit and allows many messages per connection
Unmarshal	`recv_exact` header → body → `unmarshal(body, fmt)` → dict
Before/after logging	Each SEND log stores `message` (before) and `wire` (after). Each RECV log stores `wire` (before) and `message` (after)

5. Why sockets / RPC

	TCP sockets + JSON-RPC 2.0
Reason	Milestone 1 already used TCP sockets. Sockets show the low-level mechanics the course covers (connect, framing, blocking request-reply). JSON-RPC adds a standard method/params/error contract instead of ad-hoc `action` strings
Errors	JSON-RPC codes: -32600 invalid request, -32601 method not found, -32602 bad params, 404 property not found
Methods	`rpc.ping`, `property.list/get`, `availability.check`, `property.set_available`, `reservation.book/get/list`

RMI was not chosen because it is Java-specific, and the team's stack is Python.

6. Publish-subscribe design

	Python
Broker	`Broker` process :6010
Subscribe	`subscribe {topics}` on a long-lived socket
Subscribers in demo	notification-, analytics-, audit-service (3 processes)
Publish	Reservation sends `publish` → broker fans out `event` (with seq) to each subscriber → `publish.ack {delivered_to, failed}`
Decoupling / failure	Publisher does not know the subscribers. A broken socket is removed

Topic filtering is tested: a publish on `other.topic` reaches nobody.

7. Group communication design

	Python
Coordinator	`GroupCoordinator` :6020
Members	replica-1..3, each with its own TCP listener
Membership ops	`group.join/leave/members`; every change → `group.view {view_id, members}` to all
Multicast	Coordinator sends a numbered `group.deliver` to every member and waits for `group.ack`. Replies `{seq, delivered_to, failed}`
Failure detection	Unreachable member (refused, closed or timed-out connection) is logged as an INFO `delivery ... failed` entry, removed, and a new view announced
Use	Keep property availability caches consistent across replicas

The demo kills `replica-3`. The next multicast reports `failed: ["replica-3"]` and the membership view shrinks.

8. Logging

Every SEND and RECV on every node is logged with an ISO-8601 UTC timestamp (millisecond precision, `Z`):

- `logs/messages.jsonl` (`COMM_LOG_FILE` overrides the path), plus readable stdout lines.

9. Log excerpts from real runs

All excerpts were copied from runs on 2026-10-12. The full logs are in `docs/logs/`.

9.1 Python simulation (`python -m src.communication.demo`)

```
=== 2+3+4. reservation.book -> property RPC, pub-sub, multicast =
2026-10-12T23:16:55.257Z [client] SEND  -> 127.0.0.1:6002 type=rpc.request
id=8ef12977323e fmt=json bytes=266
2026-10-12T23:16:55.266Z [client] RECV  <- 127.0.0.1:6002 type=rpc.reply
id=2a0f976ce60b fmt=json bytes=579
  booking 101: {
  "status": "accepted",
  "reservation": {
    "reservation_id": "R-fa7a220d",
    "property_id": 101,
    "property_name": "Long Beach Apartment",
    "guest_name": "Demo Guest",
    "nights": 2,
    "amount": 300,
    "status": "confirmed",
    "created_at": "2026-10-12T23:16:55.260Z"
...
```

```
=== 4b. failure: kill replica-3, multicast again ================
2026-10-12T23:16:55.571Z [client] SEND  -> 127.0.0.1:6020 type=group.multicast
id=c308b7a43408 fmt=json bytes=270
2026-10-12T23:16:55.576Z [client] RECV  <- 127.0.0.1:6020 type=group.multicast.ack
id=906842448018 fmt=json bytes=282
  multicast ack: {'group': 'property-replicas', 'seq': 2, 'delivered_to': ['replica-
1', 'replica-2'], 'failed': ['replica-3']}
2026-10-12T23:16:55.576Z [client] SEND  -> 127.0.0.1:6020 type=group.members
id=de9e1fae51be fmt=json bytes=194
2026-10-12T23:16:55.577Z [client] RECV  <- 127.0.0.1:6020 type=group.members.reply
id=5cb1ab2d7a2f fmt=json bytes=246
  group members now: ['replica-1', 'replica-2']

=== summary from logs/messages.jsonl ============================
  164 log entries, 73 SEND, 73 RECV
  notification-service   ['send confirmation email to Demo Guest for Long Beach
Apartment']
  analytics-service      ['record revenue $300 for property 101']
  audit-service          ['append audit record for R-fa7a220d']
  replica-1              ['replica state now {101: False}', 'replica state now {101:
False, 103: True}']
  replica-2              ['replica state now {101: False}', 'replica state now {101:
False, 103: True}']
```

One SEND entry (booking request) with before/after marshalling:


```
{
 "ts": "2026-10-12T23:16:55.257Z",
 "node": "client",
 "event": "SEND",
 "peer": "127.0.0.1:6002",
 "format": "json",
 "bytes": 266,
 "message": {
  "id": "8ef12977323e",
  "type": "rpc.request",
  "sender": "client",
  "receiver": null,
  "timestamp": "2026-10-12T23:16:55.257Z",
  "correlation_id": null,
  "payload": {
   "jsonrpc": "2.0",
   "method": "reservation.book",
   "params": {
    "property_id": 101,
    "guest_name": "Demo Guest",
    "nights": 2
   },
   "id": 4
  }
 },
```

```
 "wire": "
{\"id\":\"8ef12977323e\",\"type\":\"rpc.request\",\"sender\":\"client\",\"receiver\":
null,\"timestamp\":\"2026-10-12T23:16:55.257Z\",\"correlation_id\":null,\"payload\":
{\"jsonrpc\":\"2.0\",\"method\":\"reservation.book\",\"params\":
{\"property_id\":101,\"guest_name\":\"Demo Guest\",\"nights\":2},\"id\":4}}"
}
```


10. Test cases

#	Test case	Python (`python -m unittest discover -s tests -t . -v`)
TC1	JSON + XML marshal round-trip, ISO timestamp	pass
TC2	Request-reply availability.check / property.get, JSON and XML, error codes	pass
TC3	Booking → 3 subscribers notified + 3 replicas updated; double booking rejected	pass
TC4	Group join/view/multicast; failure removes member; leave	pass
TC5	Pub-sub topic filtering	pass
TC6	Every log entry ISO-8601; SEND entries have before/after	pass

Results: Python **Ran 10 tests — OK**. The raw output is in `docs/logs/python-test-results.txt`.

12. Screenshots

All screenshots come from real runs on 2026-10-12. Terminal output was rendered to PNG. The files are in `docs/milestone-2/screenshots/`.

Figure 01: Python demo, full run (10 processes)

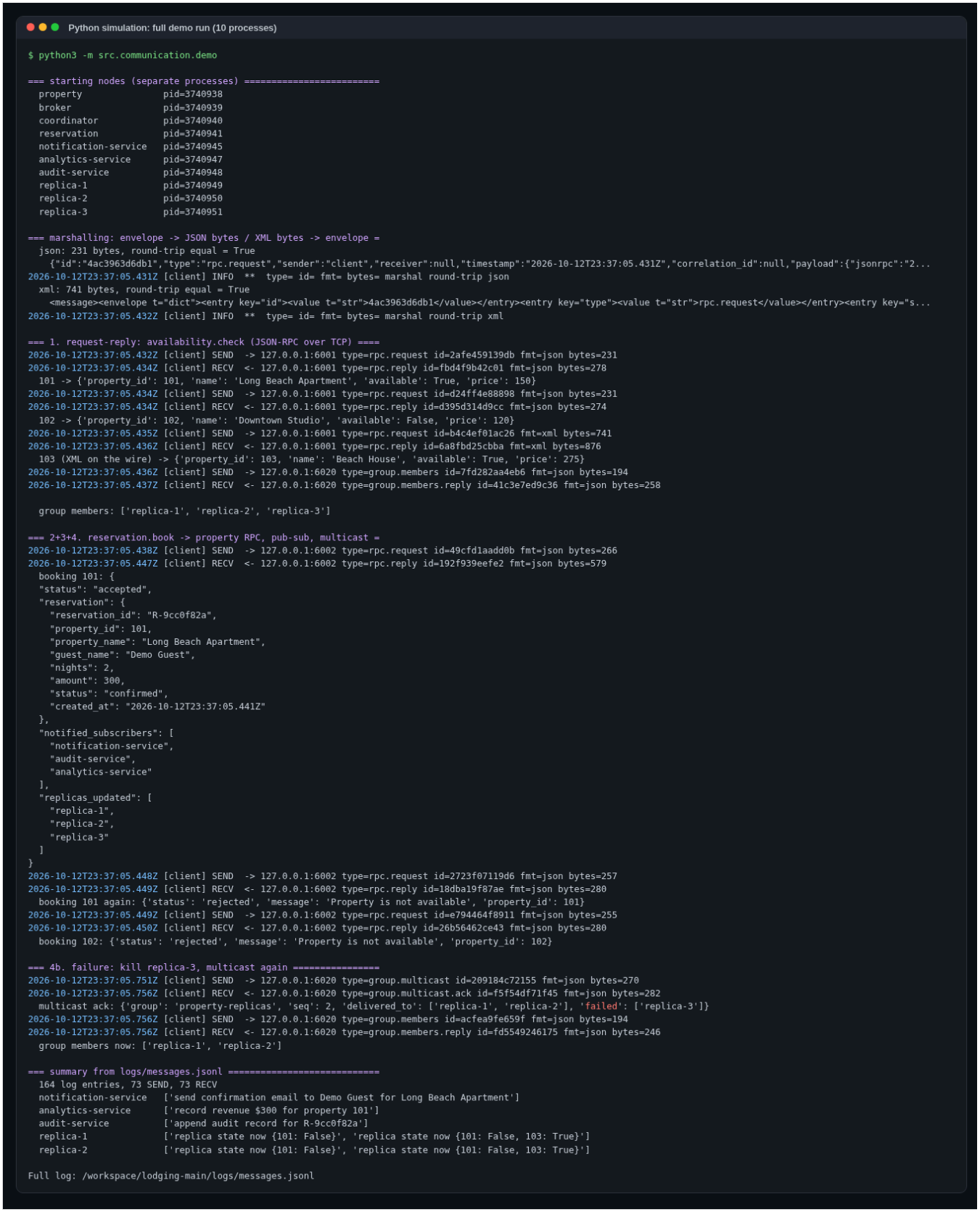


Figure 02: Request-reply: JSON-RPC over TCP

Request-reply: JSON-RPC 2.0 over TCP sockets

```
# section 1 of: python3 -m src.communication.demo
=== 1. request-reply: availability.check (JSON-RPC over TCP) ====
2026-10-12T23:37:05.432Z [client] SEND  -> 127.0.0.1:6001 type=rpc.request id=2afe459139db fmt=json bytes=231
2026-10-12T23:37:05.434Z [client] RECV  <- 127.0.0.1:6001 type=rpc.reply id=fbd4f9b42c01 fmt=json bytes=278
  101 -> {'property_id': 101, 'name': 'Long Beach Apartment', 'available': True, 'price': 150}
2026-10-12T23:37:05.434Z [client] SEND  -> 127.0.0.1:6001 type=rpc.request id=d24ff4e88898 fmt=json bytes=231
2026-10-12T23:37:05.434Z [client] RECV  <- 127.0.0.1:6001 type=rpc.reply id=d395d314d9cc fmt=json bytes=274
  102 -> {'property_id': 102, 'name': 'Downtown Studio', 'available': False, 'price': 120}
2026-10-12T23:37:05.435Z [client] SEND  -> 127.0.0.1:6001 type=rpc.request id=b4c4ef01ac26 fmt=xml bytes=741
2026-10-12T23:37:05.436Z [client] RECV  <- 127.0.0.1:6001 type=rpc.reply id=6a8fbd25cbba fmt=xml bytes=876
  103 (XML on the wire) -> {'property_id': 103, 'name': 'Beach House', 'available': True, 'price': 275}
2026-10-12T23:37:05.436Z [client] SEND  -> 127.0.0.1:6020 type=group.members id=7fd282aa4eb6 fmt=json bytes=194
2026-10-12T23:37:05.437Z [client] RECV  <- 127.0.0.1:6020 type=group.members.reply id=41c3e7ed9c36 fmt=json bytes=258

  group members: ['replica-1', 'replica-2', 'replica-3']
```

Figure 03: Marshalling before/after (JSON + XML)

Marshalling / unmarshalling: before and after (JSON + XML)

```
# demo marshalling section + entries from logs/messages.jsonl
=== marshalling: envelope -> JSON bytes / XML bytes -> envelope =
  json: 231 bytes, round-trip equal = True
    {"id":"4ac3963d6db1","type":"rpc.request","sender":"client","receiver":null,"timestamp":"2026-10-12T23:37:05.431Z","correlation_id":null,"payload":{"jsonrpc":"2...
2026-10-12T23:37:05.431Z [client] INFO  **  type= id= fmt= bytes= marshal round-trip json
  xml: 741 bytes, round-trip equal = True
    <message><envelope t="dict"><entry key="id"><value t="str">4ac3963d6db1</value></entry><entry key="type"><value t="str">rpc.request</value></entry><entry key="s...
2026-10-12T23:37:05.432Z [client] INFO  **  type= id= fmt= bytes= marshal round-trip xml


--- logs/messages.jsonl: SEND entry (message = BEFORE marshalling, wire = AFTER) ---
{
  "ts": "2026-10-12T23:37:05.432Z",
  "node": "client",
  "event": "SEND",
  "peer": "127.0.0.1:6001",
  "format": "json",
  "bytes": 231,
  "message": {
    "id": "2afe459139db",
    "type": "rpc.request",
    "sender": "client",
    "receiver": null,
    "timestamp": "2026-10-12T23:37:05.432Z",
    "correlation_id": null,
    "payload": {
      "jsonrpc": "2.0",
      "method": "availability.check",
      "params": {
        "property_id": 101
      },
      "id": 1
    }
  },
  "wire": "{\"id\":\"2afe459139db\",\"type\":\"rpc.request\",\"sender\":\"client\",\"receiver\":null,\"timestamp\":\"2026-10-12T23:37:05.432Z\",\"correlation_id\":null...
}

--- RECV entry in XML (wire = BEFORE unmarshalling, message = AFTER) ---
{
  "ts": "2026-10-12T23:37:05.435Z",
  "node": "property-rpc",
  "event": "RECV",
  "peer": "127.0.0.1:40078",
  "format": "xml",
  "bytes": 741,
  "wire": "<message><envelope t=\"dict\"><entry key=\"id\"><value t=\"str\">b4c4ef01ac26</value></entry><entry key=\"type\"><value t=\"str\">rpc.request</value></entry...
  "message": {
    "id": "b4c4ef01ac26",
    "type": "rpc.request",
    "sender": "client",
    "receiver": null,
    "timestamp": "2026-10-12T23:37:05.435Z",
    "correlation_id": null,
    "payload": {
      "jsonrpc": "2.0",
      "method": "availability.check",
      "params": {
        "property_id": 103
      },
      "id": 3
    }
  }
}
```

Figure 04: Pub-sub: booking.created to 3 subscribers

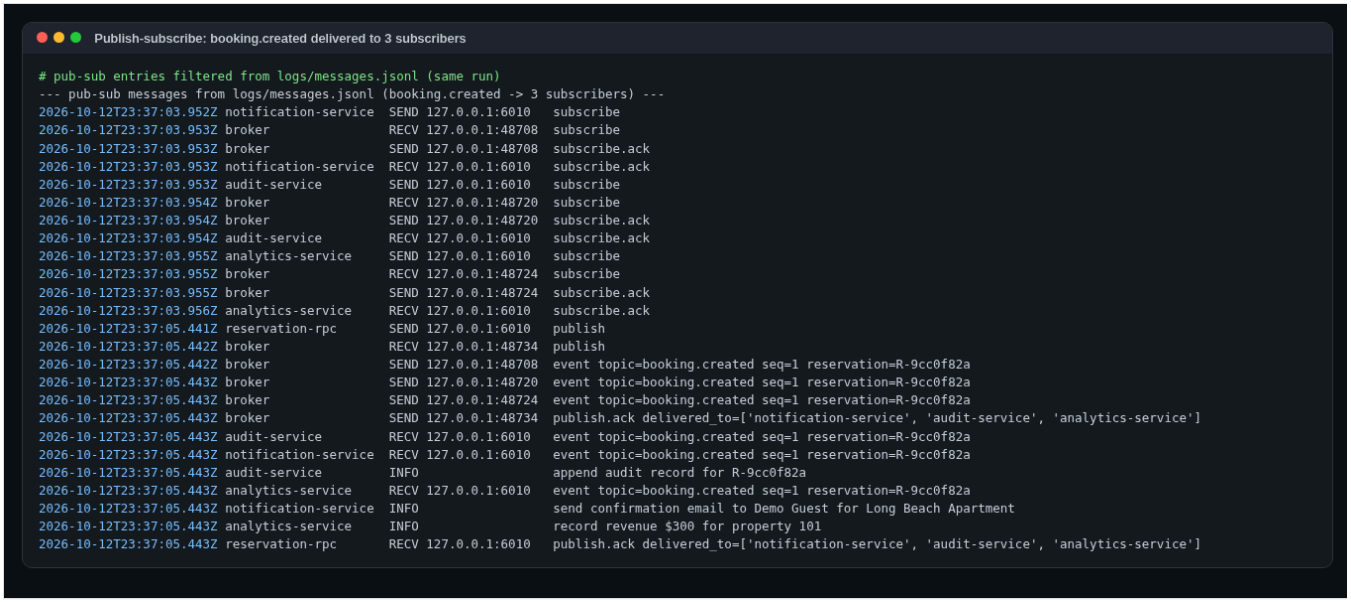


Figure 05: Group communication: multicast, acks, membership, failure

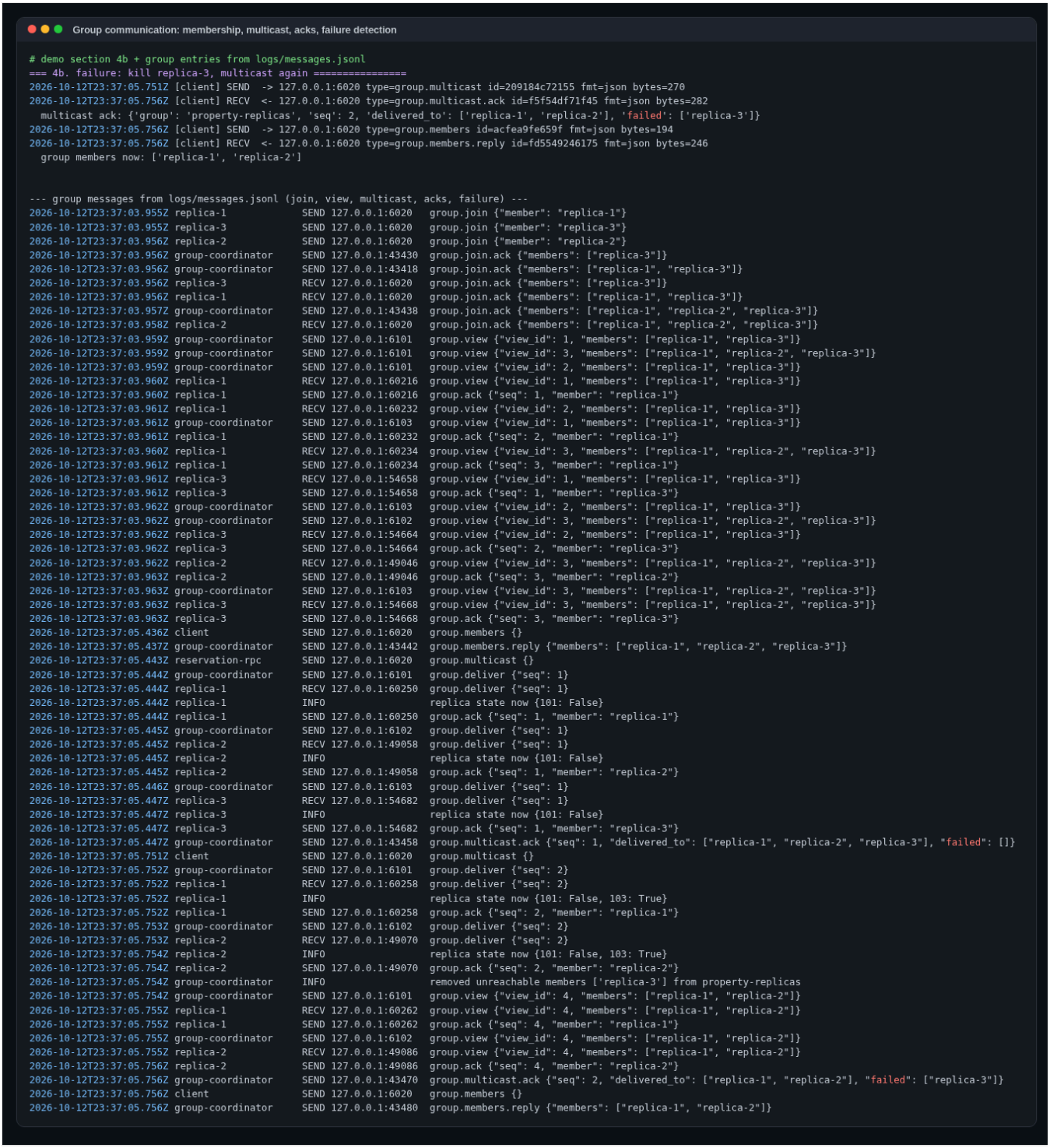


Figure 06: ISO-8601 timestamped messages.jsonl

ISO-8601 timestamped message log

```
$ head -40 logs/messages.jsonl   # long lines cut at 170 chars
{"ts":"2026-10-12T23:37:03.748Z","node":"property-rpc","event":"INFO","note":"listening on 127.0.0.1:6001"}
{"ts":"2026-10-12T23:37:03.749Z","node":"broker","event":"INFO","note":"listening on 127.0.0.1:6010"}
{"ts":"2026-10-12T23:37:03.749Z","node":"group-coordinator","event":"INFO","note":"listening on 127.0.0.1:6020"}
{"ts":"2026-10-12T23:37:03.852Z","node":"reservation-rpc","event":"INFO","note":"listening on 127.0.0.1:6002"}
{"ts":"2026-10-12T23:37:03.952Z","node":"notification-service","event":"SEND","peer":"127.0.0.1:6010","format":"json","bytes":194,"message":{"id":"608904888f12","type"...
{"ts":"2026-10-12T23:37:03.953Z","node":"broker","event":"RECV","peer":"127.0.0.1:48708","format":"json","bytes":194,"wire":"{\"id\":\"608904888f12\",\"type\":\"subscr...
{"ts":"2026-10-12T23:37:03.953Z","node":"broker","event":"SEND","peer":"127.0.0.1:48708","format":"json","bytes":244,"message":{"id":"d2c011d20d46","type":"subscribe.a...
{"ts":"2026-10-12T23:37:03.953Z","node":"notification-service","event":"RECV","peer":"127.0.0.1:6010","format":"json","bytes":244,"wire":"{\"id\":\"d2c011d20d46\",\"ty...
{"ts":"2026-10-12T23:37:03.953Z","node":"audit-service","event":"SEND","peer":"127.0.0.1:6010","format":"json","bytes":187,"message":{"id":"c9814f0e9781","type":"subsc...
{"ts":"2026-10-12T23:37:03.954Z","node":"broker","event":"RECV","peer":"127.0.0.1:48720","format":"json","bytes":187,"wire":"{\"id\":\"c9814f0e9781\",\"type\":\"subscr...
{"ts":"2026-10-12T23:37:03.954Z","node":"replica-1","event":"INFO","note":"listening on 127.0.0.1:6101"}
{"ts":"2026-10-12T23:37:03.954Z","node":"replica-3","event":"INFO","note":"listening on 127.0.0.1:6103"}
{"ts":"2026-10-12T23:37:03.954Z","node":"broker","event":"SEND","peer":"127.0.0.1:48720","format":"json","bytes":230,"message":{"id":"9e9fc7328ff9","type":"subscribe.a...
{"ts":"2026-10-12T23:37:03.954Z","node":"audit-service","event":"RECV","peer":"127.0.0.1:6010","format":"json","bytes":230,"wire":"{\"id\":\"9e9fc7328ff9\",\"type\":\"...
{"ts":"2026-10-12T23:37:03.954Z","node":"replica-2","event":"INFO","note":"listening on 127.0.0.1:6102"}
{"ts":"2026-10-12T23:37:03.955Z","node":"replica-1","event":"SEND","peer":"127.0.0.1:6020","format":"json","bytes":246,"message":{"id":"12d8b74fb9bf","type":"group.joi...
{"ts":"2026-10-12T23:37:03.955Z","node":"replica-3","event":"SEND","peer":"127.0.0.1:6020","format":"json","bytes":246,"message":{"id":"c8668dcd0a87","type":"group.joi...
{"ts":"2026-10-12T23:37:03.955Z","node":"analytics-service","event":"SEND","peer":"127.0.0.1:6010","format":"json","bytes":191,"message":{"id":"7607d3e4827d","type":"s...
{"ts":"2026-10-12T23:37:03.955Z","node":"broker","event":"RECV","peer":"127.0.0.1:48724","format":"json","bytes":191,"wire":"{\"id\":\"7607d3e4827d\",\"type\":\"subscr...
{"ts":"2026-10-12T23:37:03.955Z","node":"broker","event":"SEND","peer":"127.0.0.1:48724","format":"json","bytes":238,"message":{"id":"057f1cb8ff64","type":"subscribe.a...
{"ts":"2026-10-12T23:37:03.955Z","node":"group-coordinator","event":"RECV","peer":"127.0.0.1:43430","format":"json","bytes":246,"wire":"{\"id\":\"c8668dcd0a87\",\"type...
{"ts":"2026-10-12T23:37:03.955Z","node":"group-coordinator","event":"RECV","peer":"127.0.0.1:43418","format":"json","bytes":246,"wire":"{\"id\":\"12d8b74fb9bf\",\"type...
{"ts":"2026-10-12T23:37:03.956Z","node":"replica-2","event":"SEND","peer":"127.0.0.1:6020","format":"json","bytes":246,"message":{"id":"34d2440564a8","type":"group.joi...
{"ts":"2026-10-12T23:37:03.956Z","node":"analytics-service","event":"RECV","peer":"127.0.0.1:6010","format":"json","bytes":238,"wire":"{\"id\":\"057f1cb8ff64\",\"type\...
{"ts":"2026-10-12T23:37:03.956Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:43430","format":"json","bytes":232,"message":{"id":"1620ffa6ad09","type":"...
{"ts":"2026-10-12T23:37:03.956Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:43418","format":"json","bytes":244,"message":{"id":"ac36954711f5","type":"...
{"ts":"2026-10-12T23:37:03.956Z","node":"replica-3","event":"RECV","peer":"127.0.0.1:6020","format":"json","bytes":232,"wire":"{\"id\":\"1620ffa6ad09\",\"type\":\"grou...
{"ts":"2026-10-12T23:37:03.956Z","node":"replica-1","event":"RECV","peer":"127.0.0.1:6020","format":"json","bytes":244,"wire":"{\"id\":\"ac36954711f5\",\"type\":\"grou...
{"ts":"2026-10-12T23:37:03.957Z","node":"group-coordinator","event":"RECV","peer":"127.0.0.1:43438","format":"json","bytes":246,"wire":"{\"id\":\"34d2440564a8\",\"type...
{"ts":"2026-10-12T23:37:03.957Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:43438","format":"json","bytes":256,"message":{"id":"d992121cb9dc","type":"...
{"ts":"2026-10-12T23:37:03.958Z","node":"replica-2","event":"RECV","peer":"127.0.0.1:6020","format":"json","bytes":256,"wire":"{\"id\":\"d992121cb9dc\",\"type\":\"grou...
{"ts":"2026-10-12T23:37:03.959Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:6101","format":"json","bytes":242,"message":{"id":"634e53f9001c","type":"g...
{"ts":"2026-10-12T23:37:03.959Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:6101","format":"json","bytes":254,"message":{"id":"cc3eef66d9de","type":"g...
{"ts":"2026-10-12T23:37:03.959Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:6101","format":"json","bytes":242,"message":{"id":"a1a901f9f3aa","type":"g...
{"ts":"2026-10-12T23:37:03.960Z","node":"replica-1","event":"RECV","peer":"127.0.0.1:60216","format":"json","bytes":242,"wire":"{\"id\":\"634e53f9001c\",\"type\":\"gro...
{"ts":"2026-10-12T23:37:03.960Z","node":"replica-1","event":"SEND","peer":"127.0.0.1:60216","format":"json","bytes":204,"message":{"id":"dbd40a5fc0f2","type":"group.ac...
{"ts":"2026-10-12T23:37:03.960Z","node":"group-coordinator","event":"RECV","peer":"127.0.0.1:6101","format":"json","bytes":204,"wire":"{\"id\":\"dbd40a5fc0f2\",\"type\...
{"ts":"2026-10-12T23:37:03.961Z","node":"replica-1","event":"RECV","peer":"127.0.0.1:60232","format":"json","bytes":242,"wire":"{\"id\":\"a1a901f9f3aa\",\"type\":\"gro...
{"ts":"2026-10-12T23:37:03.961Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:6103","format":"json","bytes":242,"message":{"id":"40b687d59210","type":"g...
{"ts":"2026-10-12T23:37:03.961Z","node":"replica-1","event":"SEND","peer":"127.0.0.1:60232","format":"json","bytes":204,"message":{"id":"09c67946df4a","type":"group.ac...
```

Figure 07: Python tests (10/10)

Python tests

```
$ python3 -m unittest discover -s tests -t . -v
test_envelope_has_iso_timestamp (tests.test_communication.TC1_Marshalling.test_envelope_has_iso_timestamp) ... ok
test_json_round_trip (tests.test_communication.TC1_Marshalling.test_json_round_trip) ... ok
test_xml_round_trip_preserves_types (tests.test_communication.TC1_Marshalling.test_xml_round_trip_preserves_types) ... ok
test_availability_check (tests.test_communication.TC2_RequestReply.test_availability_check) ... ok
test_errors (tests.test_communication.TC2_RequestReply.test_errors) ... ok
test_xml_rpc_call (tests.test_communication.TC2_RequestReply.test_xml_rpc_call) ... ok
test_booking_flow (tests.test_communication.TC3_BookingPubSubGroup.test_booking_flow) ... ok
test_join_view_multicast_and_failure (tests.test_communication.TC4_GroupMembership.test_join_view_multicast_and_failure) ... ok
test_only_matching_topic (tests.test_communication.TC5_Pubsub_Topics.test_only_matching_topic) ... ok
test_every_entry_is_timestamped (tests.test_communication.TC6_Logging.test_every_entry_is_timestamped) ... ok

----------------------------------------------------------------------
Ran 10 tests in 1.781s

OK
```

13. Contribution table

Member	Contribution (communication milestone)
Member 1	*TBD*
Member 2	*TBD*
Member 3	*TBD*
Member 4	*TBD*
Member 5	*TBD*

Member	Contribution (communication milestone)
Member 6	*TBD*
Member 7	*TBD*

Fill in names and contributions before submitting.